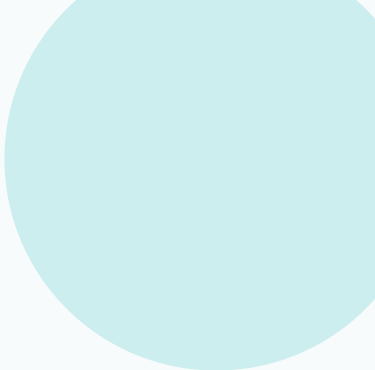




Relatório Técnico

Otimização de Rotas para Distribuição de Medicamentos e Atendimento Especializado à Mulher

Tech Challenge - Fase 2
Pós-graduação em Inteligência Artificial - FIAP

Jonas Euler Paulino de Queiroz



Sumário

Relatório Técnico - Otimização de Rotas para Distribuição de Medicamentos e Atendimento Especializado à Mulher

1. Introdução	1	9.5 Elitismo	8
2. Problema escolhido	1	10. Função fitness	8
3. Objetivos da solução	1	10.1 Distância total	8
4. Visão geral da solução	2	10.2 Penalidade de prioridade	9
5. Arquitetura da solução	2	10.3 Penalidade de janela de horário	9
6. Diagramas de arquitetura	3	10.4 Penalidade de capacidade	9
6.1 Arquitetura geral do sistema	3	11. Simulação da rota	9
6.2 Fluxo do algoritmo genético	3	12. Visualização da rota	10
6.3 Modelo de domínio e dados	4	13. Integração com LLM	11
6.4 Fluxo de integração com LLM	5	14. Experimentos e comparação de desempenho	12
7. Modelagem dos dados sintéticos	6	15. Testes automatizados	13
8. Representação genética	7	16. Impacto social e aplicabilidade	14
9. Operadores genéticos	7	17. Considerações éticas	14
9.1 População inicial	8	18. Protocolos de segurança implementados	15
9.2 Seleção	8	19. Limitações	15
9.3 Crossover	8	20. Trabalhos futuros	15
9.4 Mutação	8	21. Conclusão	16

1. Introdução

Este relatório apresenta o desenvolvimento de um sistema de otimização de rotas para distribuição de medicamentos e atendimento especializado à mulher. O projeto foi desenvolvido como parte do Tech Challenge da Fase 2 da pós-graduação em Inteligência Artificial da FIAP. A proposta escolhida foi o Projeto 2: otimização de rotas para distribuição de medicamentos e atendimento especializado à mulher. O desafio consistiu em partir de um problema clássico de caixeiro viajante, o TSP, e adaptá-lo para um cenário mais próximo de uma situação real de roteirização, considerando restrições importantes para o contexto da saúde da mulher. A solução implementada utiliza um algoritmo genético para otimizar a sequência de visitas a diferentes pontos de atendimento. Além da distância percorrida, o sistema considera prioridade dos atendimentos, janelas de horário e capacidade máxima de suprimentos do veículo. Após a otimização, a rota é exibida em uma interface com mapa interativo e também pode ser utilizada por uma LLM, via API do Ollama, para gerar relatórios operacionais em linguagem natural. O objetivo não é substituir decisões clínicas ou protocolos oficiais de atendimento, mas demonstrar como técnicas de otimização e linguagem natural podem apoiar a organização logística de atendimentos especializados.

2. Problema escolhido

O problema escolhido envolve a organização de uma rota diária para distribuição de medicamentos e realização de atendimentos especializados à mulher. O cenário simulado considera uma central de distribuição responsável por organizar a saída de um veículo que deve visitar diferentes pontos de atendimento. Esses pontos foram classificados em quatro categorias:

- 1 Emergências obstétricas;
- 2 Casos de violência doméstica;
- 3 Medicamentos hormonais;
- 4 Atendimento pós-parto.

Cada tipo de atendimento possui uma prioridade diferente. Emergências obstétricas têm prioridade máxima, seguidas por casos de violência doméstica, medicamentos hormonais e atendimentos pós-parto. A solução busca encontrar uma rota que não seja apenas curta, mas que também respeite a criticidade dos atendimentos. Por isso, o algoritmo genético foi adaptado para considerar penalidades relacionadas à ordem dos atendimentos, às janelas de horário e à capacidade de suprimentos do veículo.

3. Objetivos da solução

O objetivo principal da solução é construir um sistema capaz de otimizar rotas para atendimentos especializados à mulher, considerando restrições mais realistas do que um TSP simples. Os objetivos específicos são:

- adaptar um código base de TSP para um problema de roteirização com restrições;
- representar pontos de atendimento com dados sintéticos;
- implementar um algoritmo genético com seleção, crossover, mutação e elitismo;
- definir uma função fitness baseada em distância e penalidades;
- considerar prioridade por tipo de atendimento;

- considerar janelas de horário específicas;
- considerar capacidade máxima de suprimentos do veículo;
- visualizar a rota otimizada em um mapa interativo;
- gerar relatórios e instruções operacionais com uma LLM;
- permitir comparação entre diferentes configurações do algoritmo genético;
- manter o projeto organizado, testável e documentado.

4. Visão geral da solução

A solução foi implementada como uma aplicação Python com interface em Streamlit. O sistema carrega dados sintéticos de uma central de distribuição e 20 pontos de atendimento, executa um algoritmo genético para encontrar uma rota otimizada e exibe os resultados em uma interface visual. A aplicação também possui uma camada de integração com LLM, utilizando a API do Ollama. Essa camada não participa da otimização da rota. Ela é usada depois que a rota já foi encontrada, com o objetivo de transformar os dados técnicos em textos mais úteis para a equipe operacional, como manual de instruções e roteiro detalhado de visitas. O fluxo geral da solução é:

- 1 Carregamento dos dados sintéticos;
- 2 Criação da população inicial de rotas;
- 3 Avaliação das rotas pela função fitness;
- 4 Aplicação dos operadores genéticos;
- 5 Escolha da melhor rota;
- 1 Simulação dos horários de chegada e atendimento;
- 2 Visualização da rota no mapa;
- 3 Geração de relatórios com LLM.

5. Arquitetura da solução

O projeto foi organizado como um pacote Python modular. Essa decisão foi tomada para evitar concentração de regras de negócio em um único arquivo e facilitar a manutenção do código. A estrutura principal do projeto é:

```
ia-tech-challenge-2/
├── app.py
├── data/
│   ├── attendance_points.csv
│   └── distribution_center.csv
├── docs/
│   └── architecture/
├── experiments/
│   └── run_experiments.py
├── outputs/
├── src/
│   └── womens_health_route_optimizer/
│       ├── config/
│       ├── data/
│       ├── domain/
│       ├── llm/
│       ├── optimization/
│       ├── ui/
│       ├── utils/
│       └── visualization/
```

└ tests/

Os principais módulos são:

- config: configurações gerais do sistema;
- domain: modelos e enums do domínio;
- data: carregamento dos dados sintéticos;
- optimization: algoritmo genético, operadores, fitness e simulação;
- visualization: criação do mapa com Folium;
- ui: componentes visuais da interface Streamlit;
- llm: integração com API do Ollama e templates de prompt;
- utils: funções auxiliares;
- tests: testes automatizados.

Essa separação tornou o projeto mais organizado e facilitou a evolução da aplicação durante o desenvolvimento.

6. Diagramas de arquitetura

A modelagem da solução foi documentada por meio de quatro diagramas principais. Eles foram utilizados para orientar a implementação e também para facilitar a explicação da solução.

6.1 Arquitetura geral do sistema

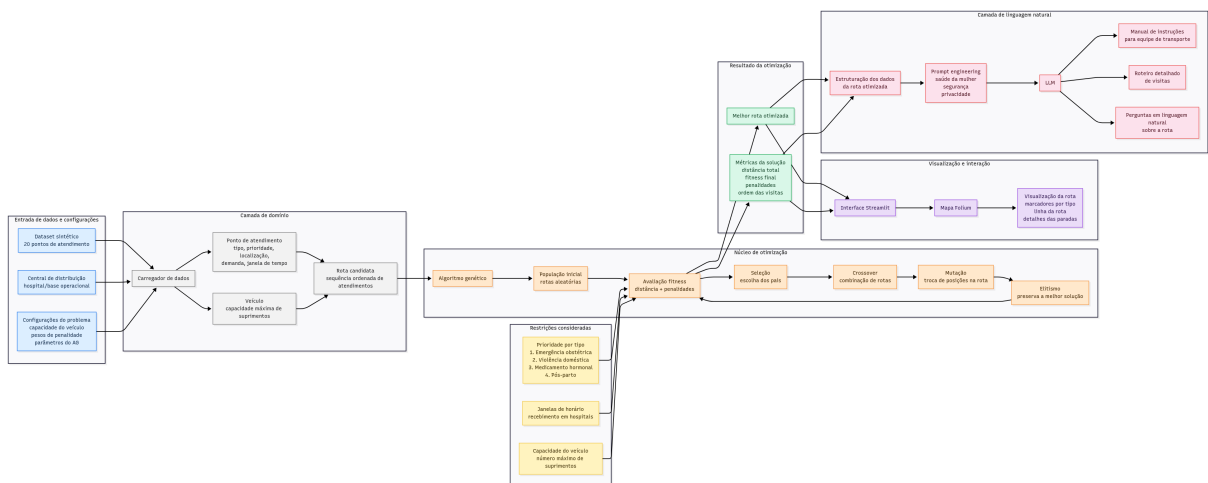


Figura - Arquitetura geral do sistema.

O diagrama de arquitetura geral apresenta os principais componentes da solução. Ele mostra a entrada de dados sintéticos, a camada de domínio, o núcleo de otimização, as restrições consideradas, a visualização em mapa e a integração com LLM. A partir dos dados e configurações, o sistema cria rotas candidatas e as avalia por meio do algoritmo genético. A melhor rota encontrada é utilizada tanto pela interface Streamlit/Folium quanto pela camada de LLM.

6.2 Fluxo do algoritmo genético

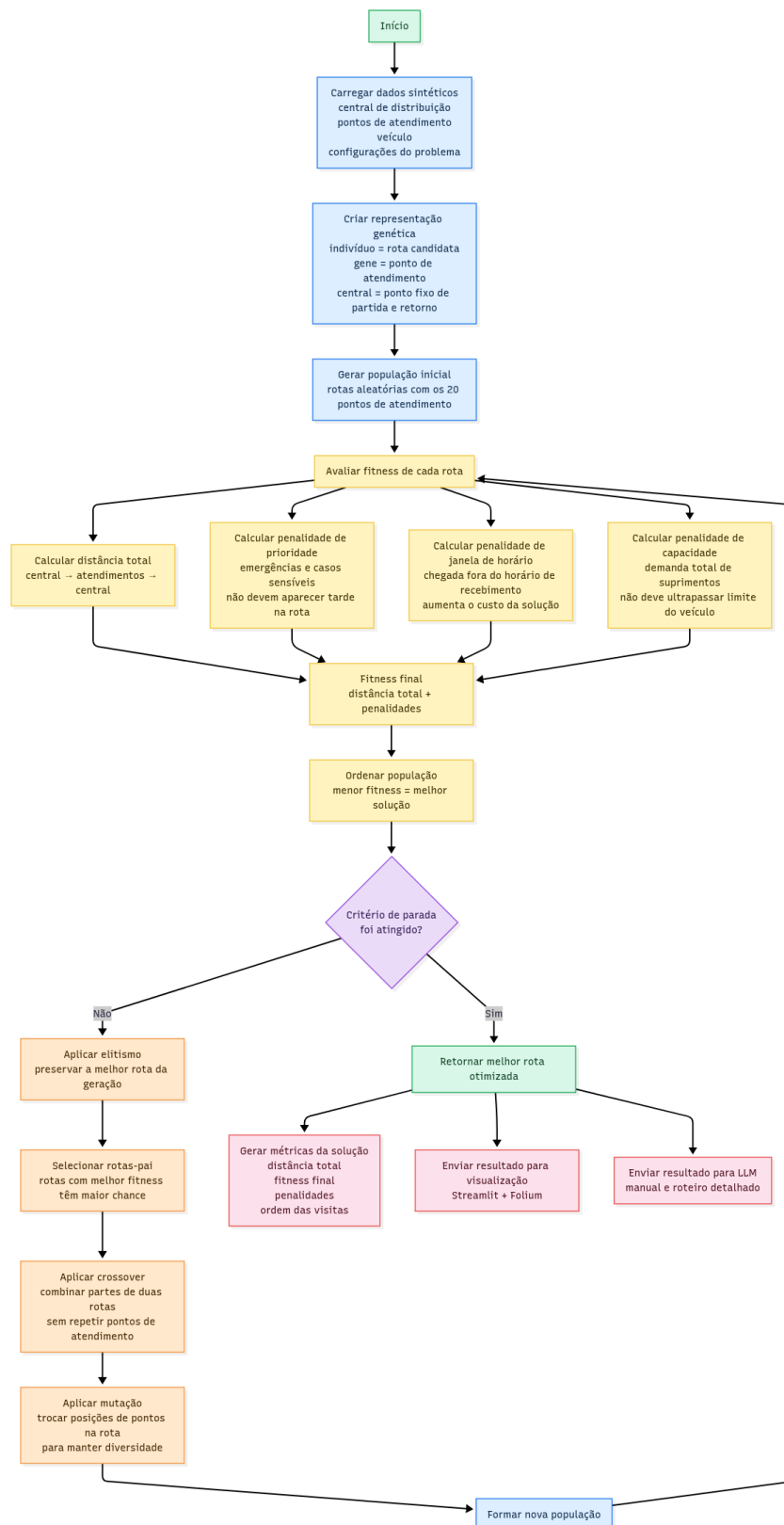


Figura - Fluxo do algoritmo genético.

O fluxo do algoritmo genético mostra o processo de otimização da rota. O sistema gera uma população inicial, avalia as rotas pela função fitness, ordena as soluções, seleciona pais, aplica crossover e mutação, preserva a melhor solução por elitismo e repete esse ciclo até atingir o número definido de gerações.

6.3 Modelo de domínio e dados

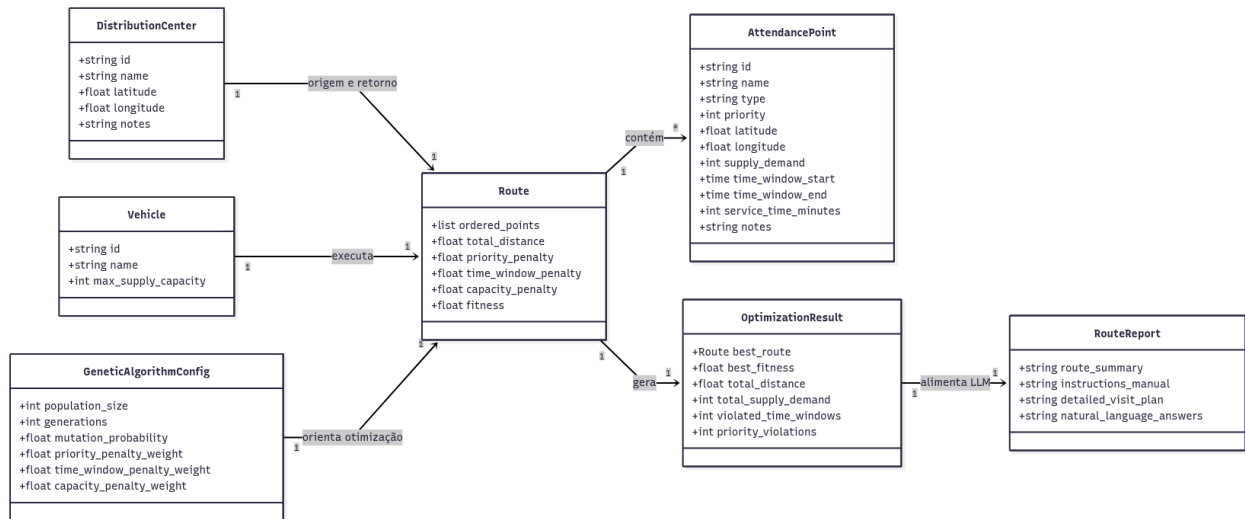


Figura - Modelo de domínio e dados.

O modelo de domínio apresenta as principais entidades do sistema, como central de distribuição, ponto de atendimento, veículo, rota, resultado da otimização e relatório gerado pela LLM. Esse modelo evidencia uma decisão importante: a central de distribuição não é tratada como gene do algoritmo. Ela é fixa, funcionando como origem e retorno da rota. Os genes são os pontos de atendimento, que são reorganizados pelo algoritmo genético.

6.4 Fluxo de integração com LLM

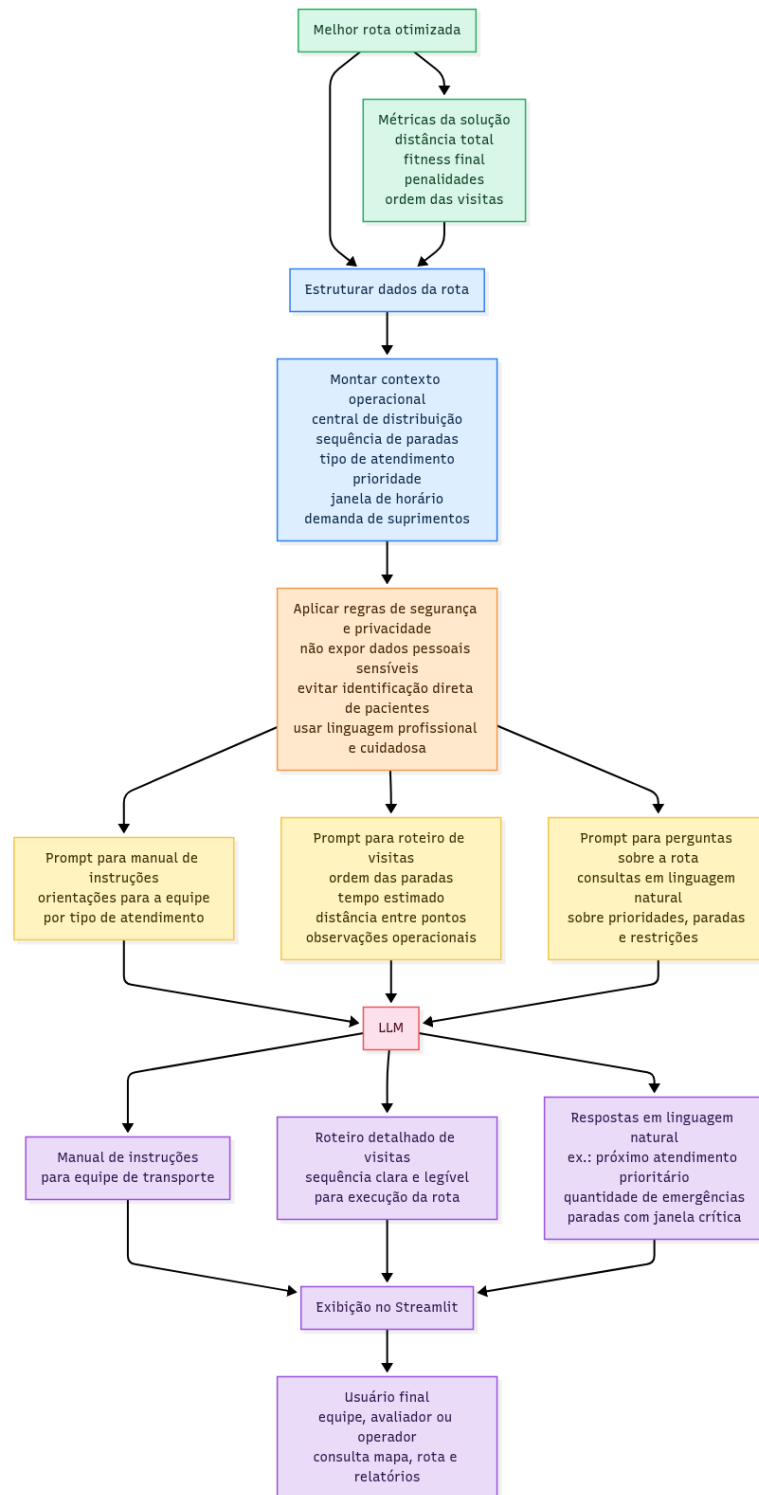


Figura - Fluxo de integração com LLM.

O fluxo de integração com LLM mostra como os dados da rota otimizada são transformados em contexto textual estruturado e enviados para a API do Ollama. A LLM não otimiza a rota. Sua responsabilidade é transformar a saída técnica do algoritmo em textos operacionais, como manual de instruções, roteiro detalhado de visitas e respostas a perguntas sobre a rota.

7. Modelagem dos dados sintéticos

Como o projeto trabalha com um cenário simulado, foram criados dados sintéticos para representar a central de distribuição e os pontos de atendimento.

O conjunto inicial contém:

- 1 central de distribuição;
- 20 pontos de atendimento.

Cada ponto de atendimento possui os seguintes campos:

id	name	type	priority
latitude	longitude	supply_demand	time_window_start
time_window_end	service_time_minutes	notes	

Os 20 pontos foram distribuídos da seguinte forma:

Tipo de atendimento	Prioridade	Quantidade
Emergência obstétrica	1	4
Violência doméstica	2	4
Medicamento hormonal	3	6
Atendimento pós-parto	4	6

A prioridade segue a regra de que valores menores representam maior urgência. Assim, prioridade 1 corresponde aos atendimentos mais críticos, enquanto prioridade 4 representa atendimentos programados com menor urgência relativa.

8. Representação genética

No algoritmo genético, cada indivíduo representa uma rota candidata. A representação adotada foi:

- gene: um ponto de atendimento;
- indivíduo: uma rota completa, composta por uma sequência ordenada de pontos;
- população: conjunto de rotas candidatas.

A central de distribuição não entra como gene. Ela é adicionada no cálculo da distância e na visualização da rota como ponto fixo de início e retorno. A rota pode ser entendida da seguinte forma: Central → Atendimento 1 → Atendimento 2 → ... → Atendimento N → Central Essa representação mantém a lógica do TSP original, mas permite que cada ponto carregue informações adicionais relevantes para o problema, como prioridade, demanda de suprimentos, tempo de serviço e janela de horário.

9. Operadores genéticos

A solução implementa os principais operadores de um algoritmo genético: população inicial, seleção, crossover, mutação e elitismo.

9.1 População inicial

A população inicial é formada por rotas aleatórias. Cada rota contém todos os pontos de atendimento, mas em uma ordem diferente. Essa etapa é importante para gerar diversidade inicial e permitir que o algoritmo explore diferentes sequências de visitas.

9.2 Seleção

A seleção dos pais é feita com base no fitness. Como o fitness representa um custo, rotas com menor fitness têm maior chance de serem selecionadas. A lógica utilizada é baseada em probabilidade inversamente proporcional ao fitness. Assim, soluções melhores tendem a gerar descendentes com maior frequência, mas soluções piores ainda podem ser selecionadas, mantendo diversidade no processo evolutivo.

9.3 Crossover

O crossover utilizado é inspirado no Order Crossover, adequado para problemas de permutação como TSP e roteirização. Esse operador preserva um segmento de um dos pais e completa o restante da rota seguindo a ordem do outro pai, evitando duplicação ou perda de pontos de atendimento. Essa escolha é importante porque, em uma rota válida, cada ponto deve aparecer uma única vez.

9.4 Mutação

A mutação é feita por troca de posição entre dois pontos da rota. Esse operador ajuda a evitar que o algoritmo fique preso rapidamente em soluções locais. Ao trocar dois pontos de posição, novas combinações podem surgir e melhorar o resultado nas gerações seguintes.

9.5 Elitismo

O elitismo preserva a melhor rota encontrada em uma geração para a próxima. Essa estratégia reduz o risco de perder a melhor solução já encontrada durante o processo evolutivo, garantindo que o algoritmo mantenha pelo menos uma solução de boa qualidade ao longo das gerações.

10. Função fitness

A função fitness é responsável por avaliar a qualidade de uma rota candidata. Neste projeto, ela foi definida como a soma da distância total da rota com penalidades relacionadas às restrições do problema. A fórmula geral é:

```
fitness = distância total  
         + penalidade de prioridade  
         + penalidade de janela de horário  
         + penalidade de capacidade
```

Como o fitness representa um custo, quanto menor o valor, melhor é a rota.

10.1 Distância total

A distância total considera o percurso completo saindo da central de distribuição, passando por todos os pontos de atendimento e retornando à central. Como os dados utilizam latitude e longitude, a distância entre os pontos é calculada com a fórmula de Haversine. Essa escolha é mais adequada do que a distância euclidiana simples em pixels, pois os pontos são representados por coordenadas geográficas.

10.2 Penalidade de prioridade

A penalidade de prioridade busca evitar que atendimentos críticos sejam deixados muito tarde na rota. A lógica considera que atendimentos com prioridade mais alta devem aparecer mais cedo. Dessa forma, uma emergência obstétrica em posição tardia gera penalidade maior do que um atendimento pós-parto na mesma posição. Essa estratégia permite que o algoritmo leve em conta a criticidade dos atendimentos, não apenas a distância percorrida.

10.3 Penalidade de janela de horário

Cada ponto possui uma janela de horário para atendimento. Durante a simulação da rota, o sistema calcula:

- horário de saída da central;
- tempo de deslocamento até o ponto;
- horário estimado de chegada;
- início efetivo do atendimento;
- tempo de serviço;
- espera, caso a equipe chegue antes da janela;
- atraso, caso a equipe chegue depois do fim da janela.

Chegar antes da janela não gera penalidade, pois a equipe pode aguardar. Chegar depois do fim da janela gera penalidade proporcional ao atraso. Essa modelagem torna a rota mais próxima de um cenário operacional real.

10.4 Penalidade de capacidade

Cada ponto possui uma demanda de suprimentos. A soma dessas demandas é comparada com a capacidade máxima do veículo. Se a demanda total ultrapassa a capacidade, a rota recebe uma penalidade proporcional ao excesso. Nesta versão, a capacidade é tratada como uma restrição flexível. Isso significa que a rota ainda pode ser gerada mesmo com excesso de demanda, mas o fitness aumenta. Essa escolha permite demonstrar o impacto da violação de capacidade sem implementar, neste primeiro momento, múltiplos veículos ou divisão automática de rotas.

11. Simulação da rota

Após a escolha da melhor rota, o sistema simula sua execução. A simulação calcula, para cada parada:

- posição na rota;
- distância do trecho anterior;
- horário estimado de chegada;
- horário de início do atendimento;
- janela de horário permitida;
- tempo de espera;
- atraso;
- demanda de suprimentos.

Essa simulação é utilizada em três partes da aplicação:

- 1 tabela operacional da interface;
- 2 pop-ups dos marcadores no mapa;
- 3 contexto enviado para a LLM.

A simulação também ajuda a validar se a rota gerada faz sentido do ponto de vista operacional, pois torna explícito onde existem atrasos e quais atendimentos estão sendo priorizados.

12. Visualização da rota

A visualização foi construída com Streamlit e Folium.

A interface apresenta:

- métricas principais da rota;
- penalidades aplicadas;
- mapa interativo;
- central de distribuição;
- marcadores coloridos por tipo de atendimento;
- linha da rota otimizada;
- tabela operacional da rota;
- gráfico de evolução do fitness;
- aba de experimentos;
- aba de relatórios com LLM.

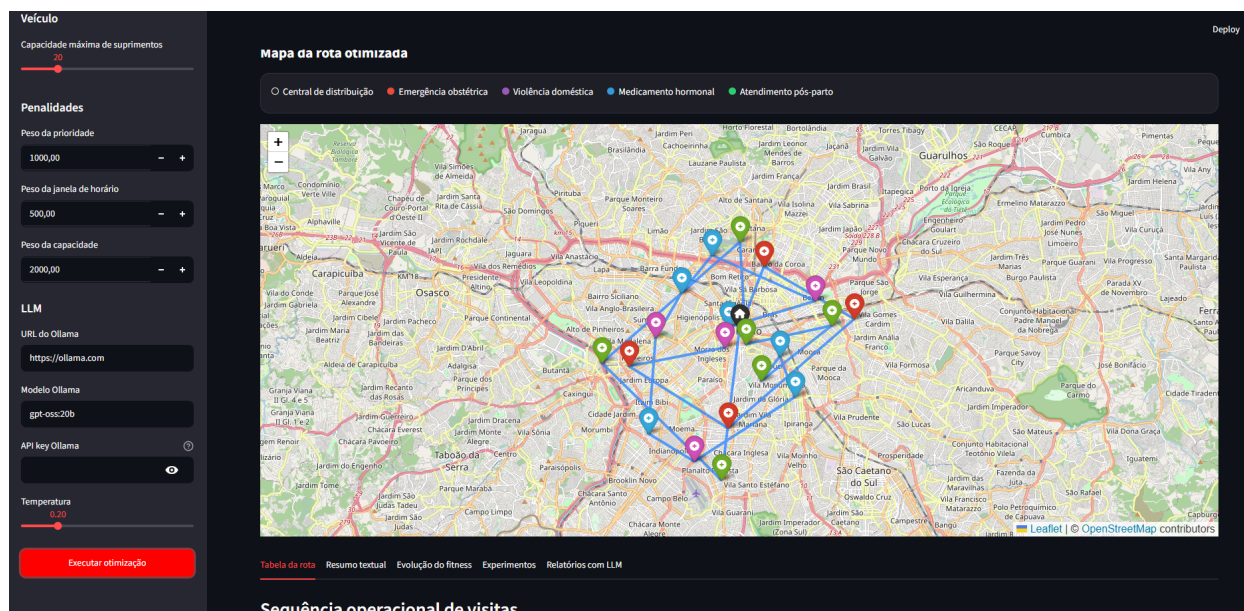


Figura - Tela principal com mapa da rota otimizada.

Os marcadores do mapa são coloridos por tipo de atendimento:

- vermelho: emergência obstétrica;
- roxo: violência doméstica;

- azul: medicamento hormonal;
- verde: atendimento pós-parto;
- preto: central de distribuição.

As linhas no mapa representam a sequência de visita definida pelo algoritmo genético. Elas não seguem ruas reais, trânsito ou malha viária. Essa foi uma decisão de escopo, pois o foco do projeto é a otimização da sequência e a incorporação de restrições operacionais, não o cálculo viário com API externa.

13. Integração com LLM

A integração com LLM foi implementada por meio da API do Ollama. A LLM é acionada depois da otimização da rota. Ela recebe um contexto estruturado com os dados da melhor rota encontrada e gera respostas em linguagem natural. A aplicação permite gerar:

- manual de instruções para a equipe de transporte;
- roteiro detalhado de visitas;
- respostas a perguntas sobre a rota.

O contexto enviado para a LLM inclui:

- central de distribuição;
- distância total;
- fitness final;
- penalidades;
- demanda total;
- sequência das paradas;
- tipo de atendimento;
- prioridade;
- horário estimado de chegada;
- início do atendimento;
- janela de horário;
- distância do trecho anterior;
- status de espera ou atraso;
- observações operacionais.

Os prompts foram mantidos em arquivos Markdown, separados do código Python. Essa decisão melhora a organização do projeto e facilita futuras alterações nos prompts. A LLM não realiza diagnóstico médico, não substitui profissionais de saúde e não decide a rota. Ela atua apenas como uma camada de apoio textual, transformando dados operacionais em instruções mais compreensíveis.

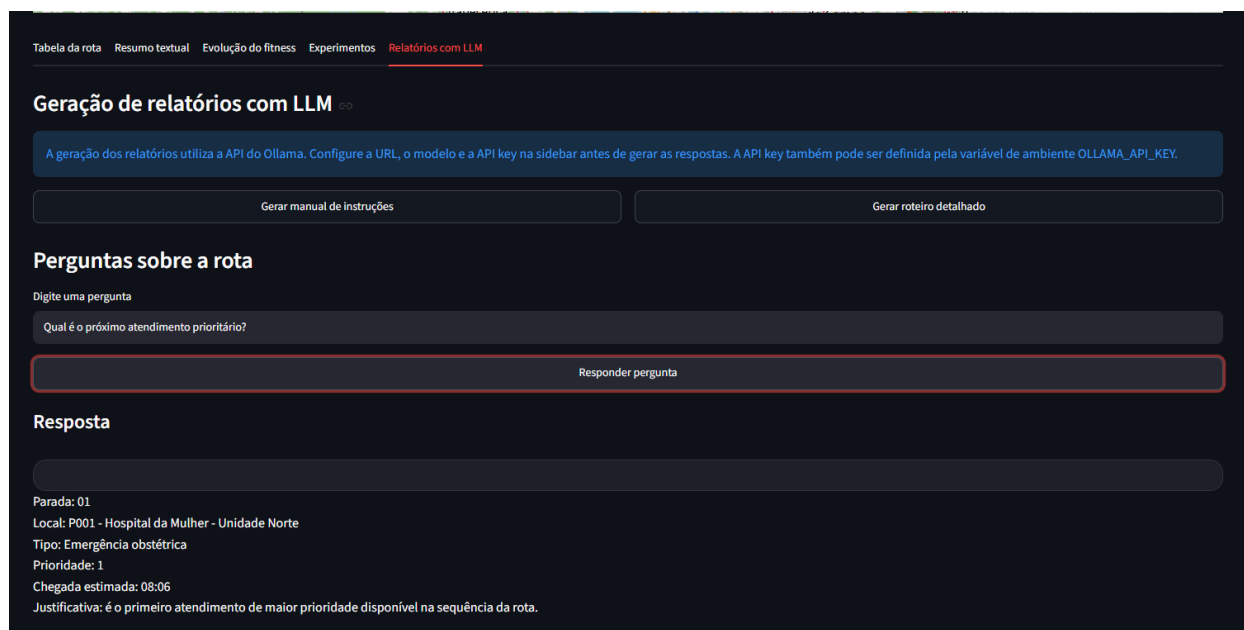


Figura - Aba de relatórios com LLM.

14. Experimentos e comparação de desempenho

Para avaliar o comportamento do algoritmo genético, foram executados experimentos com diferentes configurações de tamanho de população, número de gerações e probabilidade de mutação. Todos os experimentos utilizaram o mesmo conjunto de dados sintéticos, composto por uma central de distribuição e 20 pontos de atendimento. A capacidade do veículo foi mantida em 20 unidades de suprimentos, e os pesos das penalidades foram preservados para permitir uma comparação mais justa entre as configurações. A tabela a seguir apresenta os resultados obtidos.

Experimento	População	Gerações	Mutação	Tempo (s)	Fitness	Distância (km)	Penalidade prioridade	Penalidade janela	Penalidade capacidade
baseline_small	50	100	0.10	1.4484	580834.9854	112.8318	367000.00	173722.1536	40000.00
baseline_medium	100	200	0.20	3.0690	529248.5268	105.5306	369000.00	120142.9962	40000.00
large_population	200	300	0.20	10.9259	526925.2460	105.0279	369000.00	117820.2181	40000.00
high_mutation	100	200	0.40	3.3262	545768.1239	98.7303	369000.00	136669.3936	40000.00
no_fixed_seed	100	200	0.20	3.4833	524445.9064	104.4495	369000.00	115341.4569	40000.00

A principal métrica de comparação foi o fitness final, pois ele representa o custo total da solução, combinando distância percorrida e penalidades por prioridade, janela de horário e capacidade. O melhor fitness foi obtido no experimento no_fixed_seed, com valor de 524445.9064. Esse experimento utilizou a mesma configuração do baseline_medium, porém sem seed fixa. Isso permitiu que o algoritmo explorasse uma trajetória aleatória diferente e encontrasse uma solução ligeiramente melhor. Apesar disso, por não utilizar seed fixa, esse resultado pode variar entre execuções. Entre os experimentos com seed fixa, o melhor resultado foi o large_population, com fitness de 526925.2460. Esse experimento utilizou população de 200 indivíduos e 300 gerações, o que aumentou o espaço de busca explorado pelo algoritmo. Em comparação com o baseline_medium, houve uma pequena melhora no fitness e na penalidade de janela de horário, mas com um aumento considerável no tempo de execução, passando de 3.0690 segundos para 10.9259 segundos. O experimento baseline_small, com população de 50 indivíduos e 100 gerações, teve o pior fitness, 580834.9854. Esse resultado indica que uma população menor e menos

gerações limitaram a capacidade do algoritmo de encontrar uma rota mais adequada, especialmente em relação às janelas de horário. A penalidade de janela nesse caso foi a maior entre todos os experimentos, chegando a 173722.1536. O experimento `high_mutation`, com probabilidade de mutação de 0.40, apresentou a menor distância total, 98.7303 km. No entanto, seu fitness não foi o melhor, pois a penalidade de janela de horário foi maior do que nos experimentos `baseline_medium`, `large_population` e `no_fixed_seed`. Isso mostra que, neste problema, minimizar distância não é suficiente. Uma rota mais curta pode ainda ser pior se gerar mais atrasos ou violar restrições importantes. A penalidade de capacidade permaneceu constante em todos os experimentos, com valor de 40000.00. Isso ocorreu porque o conjunto de dados possui demanda total de 40 unidades de suprimentos, enquanto a capacidade configurada do veículo foi mantida em 20 unidades. Nesta versão do sistema, a capacidade é tratada como restrição flexível, gerando penalidade quando excedida.

Em todos os experimentos, a primeira parada foi o ponto P001, classificado como emergência obstétrica. Esse comportamento é coerente com a função fitness, pois atendimentos de prioridade máxima devem ser favorecidos nas primeiras posições da rota. De forma geral, os experimentos mostram que aumentar população e gerações tende a melhorar a qualidade da solução, mas também aumenta o tempo de execução. A configuração `baseline_medium` apresenta um bom equilíbrio entre qualidade e custo computacional, enquanto a configuração `large_population` oferece uma solução ligeiramente melhor com maior tempo de processamento. Para fins de demonstração, a configuração intermediária é adequada, pois gera bons resultados em poucos segundos e mantém reprodutibilidade quando utilizada com seed fixa.

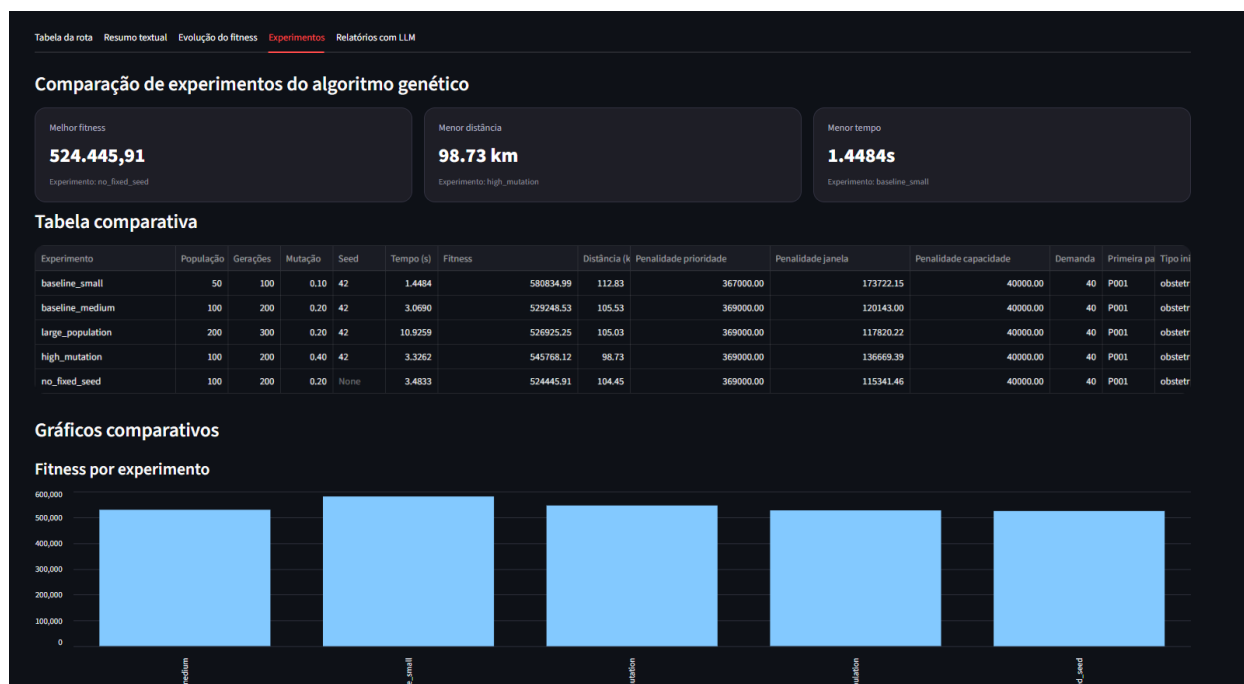


Figura - Aba de experimentos da aplicação.

15. Testes automatizados

O projeto possui testes automatizados com Pytest. Os testes cobrem partes importantes da aplicação, como:

- carregamento da central de distribuição;
- carregamento dos 20 pontos de atendimento;

- validação dos tipos e prioridades;
- cálculo de distância;
- estimativa de tempo de deslocamento;
- penalidade de capacidade;
- avaliação de rotas;
- geração da população inicial;
- crossover;
- mutação;
- simulação das paradas da rota.

Esses testes ajudam a garantir que alterações futuras não quebrem partes essenciais do sistema. Embora não cubram todas as possibilidades, eles validam a base da aplicação e aumentam a confiabilidade da entrega.

16. Impacto social e aplicabilidade

A solução proposta tem potencial de apoiar a organização logística de atendimentos especializados à mulher, principalmente em cenários nos quais existem diferentes graus de urgência e restrições operacionais.

Ao considerar prioridade por tipo de atendimento, o sistema permite que emergências obstétricas e casos de violência doméstica sejam favorecidos na sequência da rota. Isso aproxima a solução de uma lógica mais sensível ao contexto social e clínico, em vez de tratar todos os pontos como equivalentes. Além disso, a geração de relatórios com LLM pode ajudar a transformar uma saída técnica em orientações mais claras para uma equipe operacional. Isso pode ser útil em situações em que a equipe precisa compreender rapidamente a ordem das visitas, os cuidados de cada tipo de atendimento e os pontos com atraso ou maior criticidade. A aplicação é um protótipo acadêmico, mas demonstra como otimização e linguagem natural podem ser combinadas para apoiar processos logísticos em saúde.

17. Considerações éticas

O projeto trata de um contexto sensível, envolvendo saúde da mulher, violência doméstica e emergências obstétricas. Por esse motivo, algumas decisões foram tomadas para reduzir riscos éticos. A primeira decisão foi utilizar apenas dados sintéticos. Nenhum dado real de paciente foi utilizado. Os pontos de atendimento, nomes e observações foram criados apenas para simular o problema. A segunda decisão foi evitar qualquer identificação pessoal. A aplicação trabalha com locais e tipos de atendimento, mas não inclui nomes de pacientes, documentos, telefones, endereços pessoais ou informações clínicas individuais. A terceira decisão foi limitar o papel da LLM. A LLM não realiza diagnóstico, não recomenda condutas médicas e não substitui protocolos oficiais. Ela apenas transforma dados operacionais da rota em textos de apoio. Também foram incluídas instruções nos prompts para que a LLM utilize linguagem profissional, não invente informações e respeite o caráter sensível de atendimentos relacionados à violência doméstica, pós-parto e emergências obstétricas. A solução deve ser entendida como apoio logístico e operacional, não como ferramenta de decisão clínica.

18. Protocolos de segurança implementados

Alguns cuidados foram considerados na modelagem e na geração dos relatórios:

- não uso de dados reais;
- não exposição de dados pessoais;
- observações operacionais sem identificação individual;
- prompts com instruções de privacidade;
- linguagem cuidadosa para casos de violência doméstica;
- separação entre otimização logística e decisão médica;
- uso da LLM apenas após a rota já estar otimizada;
- possibilidade de configurar API key sem fixá-la no código.

Esses cuidados ajudam a reduzir riscos de exposição indevida de dados e reforçam que a solução tem caráter acadêmico e operacional.

19. Limitações

A versão atual possui limitações importantes. A primeira limitação é que a rota visualizada no mapa não segue ruas reais. As linhas representam apenas a sequência de visita entre os pontos. O sistema não utiliza trânsito, sentido das vias ou tempo real de deslocamento. A segunda limitação é o uso de um único veículo. Como a demanda total dos pontos é maior do que a capacidade padrão configurada, a capacidade é tratada como penalidade. Em uma versão mais avançada, o ideal seria dividir automaticamente a rota entre múltiplos veículos. A terceira limitação é que os dados são sintéticos. Eles servem para simular o problema, mas não representam uma operação real. Também não foram implementados banco de dados, autenticação de usuários, controle de acesso, persistência histórica das rotas ou integração com sistemas hospitalares. Por fim, a qualidade dos textos gerados depende do modelo LLM utilizado e dos prompts fornecidos.

20. Trabalhos futuros

Como evoluções futuras, podem ser implementadas:

- suporte a múltiplos veículos;
- divisão automática de rotas por capacidade;
- diferentes tipos de veículos;
- integração com APIs de roteamento real, como OSRM, OpenRouteService ou Google Directions;
- cálculo de tempo considerando ruas e trânsito;
- restrições de segurança por região e horário;
- persistência dos resultados em banco de dados;
- autenticação de usuários;
- exportação dos relatórios em PDF;
- histórico de rotas e atendimentos;

- implantação em nuvem;
- monitoramento e logging;
- infraestrutura como código.

Essas melhorias aproximariam a solução de um ambiente real de uso.

21. Conclusão

O projeto demonstrou a adaptação de um problema clássico de TSP para um cenário mais realista de roteirização com restrições. A solução implementada utiliza algoritmo genético para encontrar uma sequência de atendimentos considerando distância, prioridade, janela de horário e capacidade do veículo. A aplicação também inclui visualização em mapa, tabela operacional, simulação de horários, testes automatizados, experimentos comparativos e integração com LLM via API do Ollama. Os resultados dos experimentos mostram que a melhor rota não é necessariamente a menor em distância. Em alguns casos, rotas mais curtas tiveram fitness pior por gerarem maiores penalidades de janela de horário. Esse comportamento reforça que o sistema está considerando as restrições do problema, e não apenas resolvendo um TSP simples. Apesar das limitações, a solução atende ao objetivo de demonstrar como algoritmos genéticos e LLMs podem ser aplicados em um problema de logística voltado à saúde da mulher, respeitando princípios de privacidade, uso de dados sintéticos e cuidado com contextos sensíveis.